

Project Report: Tomasulo Functional and Timing Simulator

Kaushik Vada

1. Introduction

For this project I built a functional and timing simulator for an out-of-order processor using Tomasulo's algorithm. The processor is non-speculative and has no reorder buffer. It supports a small RISC-V-like instruction set and models reservation stations, register renaming through RS tags, functional unit latencies, CDB arbitration, and branch stalls. At the end of simulation it prints out the final register and memory state along with cycle and stall counts.

The main goal was to get the timing right – making sure instructions don't execute too early, the CDB only broadcasts once per cycle, and branch fetch stalls work correctly.

2. Architecture Modeled

The processor is 32-bit with separate instruction and data memory, fetch width 1, issue width 1, and a single CDB. The supported instructions are:

- fld
- fsd
- add
- addi
- fadd
- fmul
- fdiv
- bne

Functional unit configuration (from the spec):

Unit	Latency	Reservation Stations	Supported Ops
INT	1	4	add, addi
LS	2	3	fld, fsd
FPADD	3	3	fadd
FPMUL	4	2	fmul
FPDIV	6	1	fdiv
BU	1	1	bne

The processor is non-speculative, so when a **bne** is fetched, fetch stalls immediately and nothing else is fetched until the branch resolves.

3. Simulator Organization

I kept everything in one Python file since that seemed easiest to submit and grade. The code is split into these pieces:

- `parse_file()`: two-pass parser – first pass collects memory ints and labels, second pass decodes each instruction now that all label addresses are known
 - `Instruction`: stores the parsed fields for one instruction (opcode, registers, immediate, branch target, etc.)
 - `RSEntry`: one reservation station slot – holds V_j/V_k , Q_j/Q_k , issue order, execution state, and result
 - `FunctionalUnit`: tracks what's currently executing in one pipeline (current tag and remaining cycles)
 - `Simulator`: the main simulation class; has a method for each pipeline stage and runs the cycle loop
 - `main()`: parses command-line args and prints the final output
- Keeping parsed instructions separate from the dynamic RS state made it easier to reset and debug things.

4. Key Data Structures

The main data structures I used:

- Integer register file: $R0$ – $R31$, stored as Python ints with signed 32-bit wraparound on writes
- FP register file: $F0$ – $F31$, stored as Python floats
- Register status tables: one for int, one for FP – each entry is either `None` (register has current value) or an RS tag (some instruction is still writing it)
- Memory: a Python dict mapping byte addresses to float values; anything not in the dict reads as 0.0
- Instruction list: the parsed program indexed by PC
- Decode queue: a list used as a FIFO with max size NI
- RS banks: a dict mapping FU type to a list of `RSEntry` objects
- FU states: one `FunctionalUnit` per FU class
- Label table: maps label strings to instruction indices

5. Pipeline and Cycle Behavior

Getting the cycle ordering right was probably the trickiest part. The order within each cycle is:

1. Writeback and CDB broadcast
2. Issue from decode queue into a free RS slot
3. Fetch one new instruction
4. Execution start (pick oldest ready instruction for each idle FU)
5. Execution advance (tick counters, finish if done)

The key timing rules I enforced:

- Fetched in cycle $N \Rightarrow$ can issue no earlier than $N + 1$
- Issued in cycle $N \Rightarrow$ can start executing no earlier than $N + 1$
- Execution done in cycle $N \Rightarrow$ CDB eligible starting $N + 1$
- CDB value captured in cycle $N \Rightarrow$ can start executing no earlier than $N + 1$
- Branch resolves in cycle $N \Rightarrow$ fetch resumes in $N + 1$

Without these rules you get same-cycle chaining bugs where a value completes, broadcasts, wakes a dependent, and the dependent starts executing all in one cycle, which is wrong.

6. Hazard Handling

RAW hazards: when an instruction issues, if its source register has a pending writer, I store that writer's RS tag in Qj or Qk. When the CDB broadcasts, all RS entries watching that tag capture the value and clear their Q field.

WAW hazards: handled by the register status table. A younger write just overwrites the tag in the table. When an older instruction finishes and tries to write back, it first checks if the table still has its tag – if not, it skips the write.

WAR hazards: not really an issue with Tomasulo because operand values are captured at issue time, so later writes to the source register don't affect an already-issued instruction.

Structural hazards: naturally enforced by the finite RS banks (stall if no free slot), single FU per class, and single CDB.

7. Branch Handling

When `bne` is fetched, I immediately set a `fetch_stalled` flag. Nothing else gets fetched until the branch finishes executing in the BU. When it completes, I update the PC (either the target label or fall-through) and clear the stall flag, so fetch resumes the next cycle.

Since nothing is ever fetched past an unresolved branch, there's no need to flush anything.

8. CDB Arbitration Policy

Only one instruction can broadcast per cycle. I collect all RS entries that are `waiting_for_cdb` and whose `completed_cycle` is strictly less than the current cycle (so same-cycle completions have to wait). Among those, the one with the lowest issue order wins.

I validated this with `hazard_test.dat` where a load and an add finish in the same cycle – the load (issued first) wins the bus.

9. Benchmark Methodology

I ran three benchmarks:

- `prog.dat`: the provided loop benchmark
 - `hazard_test.dat`: tests CDB arbitration, RAW chains, and FP load-use latency
 - `branch_test.dat`: tests branch stall behavior with a small loop
- For the bonus, I also ran `prog.dat` with `NI=4`, `NI=16`, and all RS banks set to size 2.

10. Results for the Provided Benchmark

Default configuration results:

- Total cycles: 48
 - Issue stall events: 6
- Final memory state:
- `Mem[124]` = 128.0
 - `Mem[116]` = 63.0
 - `Mem[108]` = 195.0

These match the expected loop computation where each iteration does `memory[R2] = memory[R1]`

* `12 + memory[R2]`:

- Iteration 1: $10 \times 12 + 8 = 128$

- Iteration 2: $5 \times 12 + 3 = 63$
- Iteration 3: $14 \times 12 + 27 = 195$
 $R1 = 0$ and $R2 = 100$ at the end, which matches the loop exit condition.

Bonus comparison results

Configuration	Cycles	Issue Stalls	Notes
Default (NI=1, default RS)	48	6	Baseline
NI=4	48	6	No change
NI=16	48	6	No change
All RS sizes = 2	48	8	More RS pressure, same cycle count

Changing NI didn't help because the bottleneck is true data dependences and branch stalls, not the decode buffer. Shrinking all RS banks to 2 caused more stalls but didn't change total cycles because the extra pressure didn't land on the critical path.

11. Discussion Peers Section

I didn't formally collaborate with any peers on this project, so there are no peer-sourced design changes to discuss.

12. Challenges and Debugging Notes

A few things that took a while to get right:

- Same-cycle forwarding bugs – early versions let a value complete and immediately wake a dependent in the same cycle. Fixing this required tracking a `ready_cycle` for each operand and checking it strictly less than the current cycle.
- CDB contention – instructions that finish but don't win the CDB need to stay in their RS slot until they actually broadcast. Initially I was freeing slots too early.
- Memory ordering – without a ROB there's no easy way to do memory disambiguation, so I just require all older memory ops to complete their memory phase before a younger one can start.
- Branch timing – making sure fetch resumes in the cycle *after* branch resolution, not the same cycle.

The `-trace` flag and `hazard_test.dat` were really useful for catching most of these.

13. Conclusion

The simulator handles all 8 instructions, matches the expected output for the provided benchmark, and passes a broader set of tests covering RAW/WAW/structural hazards, CDB contention, memory ordering, and branch stalls. The bonus parameterization works from the command line without changing the source.